

Project workflow

1. Frame the decision

The unit of planning in this demo is SKU-day. Forecast the chosen demand horizon; compare predicted needs against on-hand stock and replenishment assumptions. The tool produces reviewable suggestions, not autonomous orders.

2. Source and validate data

Input schema is defined in docs/data-dictionary.md. The demo generates 540 daily observations for eight synthetic SKUs. Raw inputs and an example end-to-end output snapshot are bundled under data/. Alternatively, pass sales and product CSVs to the CLI or upload them in Streamlit. Validate types, uniqueness, date coverage, missing products, impossible prices, and return/cancellation semantics.

3. Transform

Clean transaction rows, preserve negative returns for audit, net returns against same-day sales, and floor resulting demand at zero. Build a complete SKU-date panel so no-sale days are explicit zeros. Use dimensions and SQL from sql/ when loading a warehouse.

4. Explore and engineer features

Inspect daily demand patterns, category totals, SKU volatility, and zero-demand frequency using eda.py. Lags and trailing averages in features.py are shifted so the target day is never included. Keep all features available at forecast time; do not use actual future inventory or realized promotion outcomes.

5. Train and evaluate

Four models are compared: last observation, seven-day seasonal naive, seven-day trailing mean, and HistGradientBoostingRegressor. Each SKU is evaluated on the final contiguous holdout window. This simple single-origin evaluation is a baseline; production work should use rolling-origin backtests and multiple seasons.

6. Translate forecast to policy

The service calculates service-level Z, safety stock, reorder point, EOQ, days of supply, risk, and MOQ-aware suggested quantity. Missing purchase orders and stock movements mean the output cannot be used as a live stock ledger.

7. Present and publish

Use the Streamlit dashboard for interactive decisions; FastAPI exposes a read-only synthetic-demo endpoint; SQL is a schema/query starter. The publication builder generates PDF guides/reports, an Excel data dictionary, a 12-slide presentation, and SVG diagrams. Run tests, review generated metrics, and clearly disclose synthetic data in every presentation.

Local commands

```
pip install -r requirements.txt
```

```
pip install -e .
```

```
python -m demand_inventory.cli --horizon 30 --output-dir data/processed
```

**or add --sales-csv data/raw/sales.csv --products-csv
data/raw/products.csv**

`python -m pytest`

`streamlit run dashboard/app.py`

`uvicorn api.main:app --reload`

`docker compose up --build`

Generate the PDF/Excel/PowerPoint/SVG publication package with `pip install -e ".[publication]"` followed by `python scripts/build_publication_assets.py`.

Create a portable source/data/reports bundle with `python scripts/package_project.py`; it writes the ZIP next to the repository and emits a SHA-256 sidecar.

Suggested production extensions

Extend CSV ingestion with managed data contracts, store lead-time demand and purchase orders, version forecasts, add rolling-origin evaluation, prediction intervals, monitoring/alerting, access controls, audit logging, approval workflow, and integration tests against a managed database. Validate replenishment decisions with operations and finance before deployment.